

MOL — Diagnóstico Sistémico

Versión: 1.0

Fecha: 2026-03-29

Propósito: Contexto completo para planificación de implementación con Claude Code

Origen: Sesión de análisis arquitectónico profundo basada en consultas al sistema real

El principio que une todo

El sistema extrae conocimiento del mercado argentino pero no lo acumula.

El MOL tiene un ciclo de corrección maduro — detecta errores, crea reglas, reprocesa. Pero le falta el ciclo de aprendizaje — acumular, formalizar, promover, retroalimentar. El conocimiento entra al sistema pero no circula entre sus partes.

La consecuencia práctica: el sistema procesa pero no reflexiona sobre sí mismo. Esa reflexión hoy la hace el operador humano — manualmente, sin avisos, mirando dashboards que nadie abre automáticamente.

Estado real del sistema

Lo que existe y funciona bien

- Pipeline orquestado de 8 pasos con sub-pasos y loop interno de re-procesamiento NLP
- `id_oferta` consistente e inmutable desde scraping hasta Supabase
- `run_id` que viaja por `ofertas_esco_matching` y `validation_errors`
- `pipeline_runs` con métricas, versiones de código, snapshot de configs y delta vs run anterior
- `validation_errors` con severidad, escalado y flag de corrección
- Centro de Control con semáforos por fase y alertas con acciones
- `compare_runs.py` con delta de métricas y cambios de ISCO entre runs
- `pipeline_local_status_server.py` (puerto 8099) con métricas en tiempo real
- Skills Intelligence completo: Perfil Argentino, emergentes, versionado con rollback
- `training_pairs.json` (657 pares) con linaje completo — el artefacto más rico del sistema
- `v_reglas_efectividad` — vista SQL que mide cuántas veces se aplicó cada regla

Lo que existe pero está desconectado

- `compare_runs.py` existe pero es manual — nadie lo ejecuta automáticamente post-run
- `v_reglas_efectividad` existe pero nadie la consulta automáticamente

- `training_pairs.json` sabe qué regla creó, pero la regla no sabe que nació de un training pair
 - Skills Intelligence detecta emergentes pero no retroalimenta al extractor
 - Emergentes aprobadas no generan training pairs ni skills_rules automáticamente
 - 15.968 ofertas validadas que no alimentan el Gold Set
-

Los números que importan

42.190 ofertas con tareas extraídas
245.950 tareas individuales procesadas
164.656 tareas únicas en el vocabulario argentino

De esas 42.190 ofertas:

31.537 (75%) → tienen skills ESCO → matching completo

10.653 (25%) → 0 skills ESCO

└─ 5.415 → NLP OK, gate OK, matching NUNCA CORRIÓ (deuda de procesamiento)

└─ 3.661 → tienen ISCO asignado pero sin skills (clasificadas por título/regla)

└─ 1.577 → tareas muy cortas, probablemente ruido

De las 3.661 clasificadas sin skills:

1.639 (45%) → clasificadas por semántico solo del título

1.502 (41%) → clasificadas por regla de negocio (keyword en título)

483 (13%) → dual coinciden

Fuentes de skills declaradas que nunca llegan a ESCO:

skills_tecnicas_list	322.222 valores	10.997 únicos
tecnologias_list	39.697 valores	8.702 únicos
herramientas_list	22.716 valores	6.716 únicos
soft_skills_list	~31K ofertas	nunca matcheada

Total combinado ~384.635 valores 23.157 únicos
que el sistema tiene pero no conecta con ESCO ni el Perfil Argentino

De las ~18.000 tareas individuales que no produjeron skill:

~70-75% Tipo B → el concepto existe en ESCO, el vocabulario diverge

~15-20% Tipo C → competencias genuinamente nuevas (IA, herramientas post-2021)

~10% Ruido → no son tareas reales

El caso de IA como Tipo C puro

ESCO tiene 3 skills de IA en su catálogo de 14.247.
El mercado argentino pide ~50 competencias reales de IA en 712 ofertas.

Tecnologías declaradas:	Tareas reales:
ChatGPT, LangChain,	Implementar RAG
PyTorch, TensorFlow,	Diseñar arquitectura ML
RAG, LLMs, Copilot,	Construir modelos con LangChain
OpenAI, pgvector,	Desarrollar agentes de IA
Multi-agentes...	Prompt engineering y fine-tuning

ESCO no tiene nada sobre LLMs, RAG, prompt engineering, agentes de IA ni herramientas posteriores a ~2021. Estas competencias son Tipo C genuino — no se resuelven con sinónimos, requieren ampliar el catálogo. Es el dominio de mayor crecimiento en demanda laboral y el sistema no puede verlo.

Diagnóstico por categoría

P1 — Visibilidad operativa

Sin esto el operador no sabe qué está pasando

Item	Problema	Impacto
P1.1	No hay reporte consolidado post-run	El operador tiene que abrir 4 lugares distintos para entender qué pasó
P1.2	Las alertas son pull, no push	El modelo de embeddings se cayó, el sistema usó fallback, nadie lo supo
P1.3	Sync VPS→Local sin auditoría	Único eslabón del pipeline sin trazabilidad comparable al resto
P1.4	<code>pipeline_stage</code> no es explícito	El estado de cada oferta se infiere por presencia de datos en tablas — frágil

Caso concreto que falló: El modelo BGE-M3 se cayó silenciosamente. El matching degradó al fallback semántico sin skills. El sistema procesó igual y nadie se enteró porque las alertas solo son visibles si alguien entra al dashboard.

P2 — Registro de lo que se pierde

Sin esto no se puede conectar ni mejorar nada

Item	Problema	Impacto
P2.1	Tareas fallidas sin registro	60K tareas intentaron matchear ESCO y fallaron — no quedó rastro del intento, score máximo, ni skill más cercana
P2.2	Fuentes declaradas sin matchear	384K valores de skills, tecnologías y herramientas declaradas por las empresas nunca se comparan con ESCO
P2.3	Deuda de procesamiento	5.415 ofertas con NLP OK y gate OK que nunca tuvieron matching. Correr <code>--only-pending</code> las resuelve operativamente, pero siguen sin skills

Consecuencia: El sistema sabe qué matcheó pero no sabe qué intentó y falló. Sin esa señal no puede detectar que una ocupación tiene vocabulario argentino que ESCO no cubre, ni alertar que un run tuvo tasa de extracción baja.

P3 — Circulación de conocimiento

Sin esto el sistema corrige pero no aprende

Item	Problema	Impacto
P3.1	300 reglas de negocio sin linaje	Ninguna regla sabe por qué existe — sin fecha, autor, issue que la motivó, ni cuántas ofertas la generaron
P3.2	Gold Set estático	49 casos que no crecen vs 15.968 validaciones humanas que no alimentan nada
P3.3	training_pairs desconectado de reglas	El par sabe que generó R218, pero R218 no sabe que nació de ese par
P3.4	v_reglas_efectividad sin consumo automático	Existe, mide, nadie la consulta

La paradoja central: `training_pairs.json` es el único artefacto con linaje completo — issue, autor, justificación, regla creada. Pero esa información no fluye de vuelta a la regla. El artefacto más rico del sistema está desconectado del artefacto más poderoso.

P4 — El observatorio real

Con esto el MOL trasciende el clasificador

Item	Problema	Impacto
P4.1	Tipo C no puede entrar al Perfil Argentino	Solo entran skills con URI ESCO. Las competencias genuinamente argentinas sin equivalente ESCO son invisibles para siempre
P4.2	Skills Intelligence no retroalimenta al extractor	El extractor busca en 14.247 skills ESCO crudas sin usar el vocabulario curado que ya existe
P4.3	Emergentes sin conexión downstream	Una emergente aprobada no genera training pair ni skills_rule automáticamente
P4.4	No hay promoción de patrones	Corrección repetida N veces no sugiere regla. Regla con alta precisión no sugiere training pair
P4.5	Base de datos para fine-tuning incompleta	El fine-tuning planificado (embeddings BGE-M3 + LLM Qwen) necesita datos limpios con linaje — hoy el 25% de las tareas no tiene registro del intento

La arquitectura de cuatro capas

El sistema actual tiene tres capas operativas. Falta la cuarta que las conecta.

ADQUISICIÓN

Scraping → Sync → Dedup → Priorización

Gap: sync manual sin auditoría

PROCESAMIENTO

NLP → Matching → Validación → Auto-corrección

Gap: pipeline_stage implícito, tareas fallidas sin registro

PRESENTACIÓN

Supabase → Dashboard analistas

Gap: sync manual, analistas no ven nada hasta que el operador lo dispara

REFLEXIÓN (no existe como sistema)

4.1 Observabilidad activa

Reporte post-run automático que consolida todo

Alertas push cuando algo degrada

4.2 Linaje de conocimiento

Cada regla sabe por qué existe

El vínculo training_pair → regla → origen se cierra bidireccionalmente

4.3 Gold Set dinámico

Cada validación humana es candidata al Gold Set

Crece con el sistema en lugar de quedar congelado

APRENDIZAJE (planificado, sin base sólida todavía)

Track A: Fine-tuning embeddings BGE-M3

Dataset: pares (tarea_argentina, skill_ESCO_correcta)

Necesita: P2.1 resuelto + P3.2 resuelto

Track B: Fine-tuning LLM Qwen

Dataset: pares (texto_crudo, extracción_correcta)

Necesita: training_pairs con linaje completo + P3.1 resuelto

La secuencia de construcción

Cada nivel habilita el siguiente. No se puede saltar ninguno.

NIVEL 1 – P1 Visibilidad

Reporte post-run consolidado

Alertas push

Auditoría sync VPS→Local

pipeline_stage explícito

→ Sin esto el operador sigue ciego

→ Es el prerequisite para todo lo demás

NIVEL 2 – P2 Registro

Registrar tareas fallidas (tarea + score + skill más cercana)

Conectar fuentes declaradas con ESCO via BGE-M3

Resolver deuda de procesamiento (--only-pending)

→ Sin esto no sabés qué está fallando ni por qué

→ Habilita entender si el problema es Tipo B o Tipo C

NIVEL 3 – P3 Circulación

Linaje de reglas (por qué existe cada una)

Gold Set dinámico (crece con cada validación)

Vincular training_pairs con reglas bidireccionalmente

Activar v_reglas_efectividad automáticamente

→ Sin esto el conocimiento se genera pero no circula

→ Habilita base de datos limpia para fine-tuning

NIVEL 4 – P4 Observatorio

Tipo C → Perfil Argentino sin URI ESCO

Retroalimentar extractor con vocabulario curado

Conexión downstream de emergentes aprobadas

Promoción automática de patrones

Base de datos para fine-tuning

→ Con esto el MOL es un observatorio real del mercado argentino

→ El sistema mejora solo con supervisión humana

Decisiones de diseño que no cambian

Estas decisiones del sistema original se mantienen:

1. **LOCAL** es el centro de control, **CLOUD** es para colaboración
2. **Validación en capas: automática primero, humana para candidatos**
3. **Spec-driven development: diseñar completo antes de implementar**

4. **Separar NLP y matching:** son problemas distintos con datasets, modelos y métricas distintas — conflactarlos degrada ambos
 5. **Human validation como gate final, no bottleneck de iteración**
-

Qué NO hacer

- No construir nada nuevo hasta tener P1 resuelto — sin visibilidad no se puede evaluar el impacto de ningún cambio
 - No bajar el umbral de 0.40 sin primero registrar los intentos fallidos — sin datos no se sabe si bajar el umbral ayuda o introduce ruido
 - No escalar el fine-tuning sin tener linaje limpio — entrenar sobre datos sin provenance produce modelos que mejoran en métricas pero degradan en producción
 - No automatizar la promoción de patrones — siempre con aprobación humana en el loop
-

Contexto institucional

Sistema: MOL (Monitor de Ofertas Laborales) — OEDE, Argentina

Usuarios finales: Analistas de mercado laboral

Stakeholders: Validadores humanos, colegas con propuestas de extensión

Objetivo de precisión: 95%+ antes de liberar datos a analistas

Escala actual: 42.190 ofertas procesadas, 6 portales, pipeline bi-semanal

El MOL no es solo un clasificador. Es un **observatorio del trabajo real argentino** — documenta la brecha entre lo que ESCO dice que debería pedir una ocupación y lo que el mercado argentino realmente pide. Esa brecha es el dato más valioso para políticas de empleo y formación.

Documento generado en sesión de análisis: 2026-03-29

Basado en consultas directas al sistema real via Claude Code

Para uso como contexto en sesiones de implementación con Claude Code